

IoT Forensic Gateway:

*Automated Real-Time Forensic Gateway &
Predictive Network Intrusion Detection System*

Project Report

BS Cyber Forensics — 2nd Semester

2025 – 2026

Abstract

The proliferation of Internet of Things (IoT) devices in industrial and commercial environments has introduced a significant forensic challenge: volatile network telemetry from embedded devices disappears within moments of an incident, leaving investigators with no recoverable evidence. This paper presents the design and implementation of an Automated Real-Time IoT Forensic Gateway paired with a Predictive Network Intrusion Detection System (NIDS). The proposed system emulates three categories of industrial IoT edge nodes HVAC climate controllers, smart access-lock systems, and electrical lighting controllers using Ubuntu virtual machines. All telemetry payloads are transmitted as structured JSON frames over UDP to a centralized Kali Linux gateway node. The gateway employs asynchronous socket multiplexing via the Python select module to ingest concurrent data streams without thread-blocking, immediately applies SHA-256 cryptographic fingerprinting to each packet for tamper-evident chain-of-custody logging, and enforces a rule-based signature engine that detects out-of-bounds sensor values, port-identity mismatches, and brute-force access events. All verified and flagged events are persisted atomically to a SQLite relational database. A lightweight web dashboard built on Python's http.server module provides real-time forensic visibility through asynchronous AJAX-driven table updates. The system operates under a Zero-Trust network validation philosophy, treating every packet as potentially adversarial until identity and boundary checks pass.

Introduction

Background

The global IoT device count surpassed 15 billion active endpoints by 2024, with industrial environments representing the fastest-growing segment. Smart HVAC systems, networked access control units, and automated lighting grids are now standard components of building management infrastructure. Unlike conventional computing assets, these devices lack persistent local storage, traditional operating systems, or forensically accessible file systems. Their entire operational record exists solely within the network packets they transmit during operation.

The Forensic Gap in IoT Environments

Traditional digital forensics methodology is predicated on the ability to acquire a static disk image from a target device. This approach is fundamentally incompatible with IoT hardware. When a smart sensor is reset, powered off, or physically tampered with, any in-memory state is irreversibly lost. The network traffic generated by that device the only artifact capturing its behavioral state at the time of an incident is equally transient unless an external capture mechanism is in place. This gap between the richness of IoT behavioral data and the inability to preserve it constitutes what forensic practitioners term the IoT forensic gap.

Motivation

Existing network forensics tools such as Wireshark, Zeek, and NetworkMiner provide passive capture capabilities, but none automate the complete pipeline from raw packet reception through cryptographic sealing to structured database persistence. This project addresses that gap by engineering an end-to-end forensic ingestion pipeline that requires no modification to the monitored IoT devices, operates continuously without human intervention, and produces court-admissible evidence records with verified chain-of-custody documentation.

Scope

The system targets three classes of industrial IoT endpoints: climate control units (HVAC_Ctrl_01), physical access management nodes (SmartLock_Zone_02), and electrical load controllers (LightCtrl_Floor_01). All devices are emulated using Lubuntu virtual machines on a VMware Workstation Pro hypervisor, communicating over an isolated internal UDP network to a Kali Linux gateway host at IP address 192.168.234.128.

Literature Review

A review of existing research reveals that IoT forensics remains a discipline in active development, with no single framework addressing the complete forensic lifecycle in real time.

- Zawoad and Hasan (2015) proposed a cloud-based IoT forensics framework addressing evidence collection from cloud-connected devices. While pioneering in scope, the framework operated postincident and offered no real-time capture or cryptographic integrity verification of individual packets.
- Harbawi and Varol (2017) introduced a structured digital evidence collection model for IoT environments, covering the evidence lifecycle from identification through presentation. However, the model remained largely theoretical and provided no automated implementation or hashing mechanism to verify data authenticity.
- Meffert et al. (2017) conducted practical traffic analysis of commercial smart home devices using Wireshark and tcpdump. Their work demonstrated the feasibility of passive capture but did not address automated storage, integrity sealing, or intrusion detection within the captured stream.
- Sivaraman et al. (2015) applied Software-Defined Networking principles to profile and classify IoT traffic patterns, identifying behavioral anomalies. The work focused on anomaly identification rather than forensic preservation, and lacked persistent evidence storage.
- Apthorpe et al. (2017) demonstrated traffic fingerprinting techniques on commercial IoT devices, revealing significant privacy implications. Their passive sniffing approach produced no chain-of-custody records and offered no intrusion detection capability.

- Wu et al. (2020) proposed a blockchain-based IoT forensics system using Ethereum smart contracts for tamper-proof ledger maintenance. While cryptographically robust, the computational overhead of blockchain consensus mechanisms makes the approach unsuitable for real-time high-frequency telemetry environments.
- Nieto et al. (2018) developed a forensic readiness framework for IoT infrastructures, emphasizing proactive preparation before incidents occur. The framework was architectural in nature and did not provide an executable implementation.
- Oriwoh et al. (2013) published early foundational guidelines for IoT forensic investigation, establishing terminology and procedural principles still referenced today. The work predates modern IoT protocol diversity and lacks applicability to contemporary UDPbased sensor networks.
- Chernyshev et al. (2018) examined physical memory extraction from IoT devices as a forensic technique. This destructive approach requires physical access to the target hardware and renders the device non-operational during examination, making it unsuitable for live environment monitoring.
- Ahmad et al. (2019) analyzed MQTT protocol traffic from smart home devices, extracting forensic artifacts from message broker logs. The scope was limited to MQTT exclusively and did not generalize to raw UDP telemetry streams characteristic of industrial sensor networks.

Study	Method	Environment	Tools	Limitation	Gap Addressed
Zawood & Hasan (2015)	Cloud forensics framework	Simulated cloud-IoT	OpenStack	Post-incident only	No real-time capture
Harbawi & Varol (2017)	Evidence lifecycle model	Generic IoT testbed	Custom scripts	No integrity check	No hashing
Meffert et al. (2017)	Traffic analysis	Smart home devices	Wireshark, tcpdump	Manual process	No automation
Sivaraman et al. (2015)	Traffic profiling	Real home network	SDN controller	No persistence	No storage layer
Apthorpe et al. (2017)	Traffic fingerprinting	Commercial IoT	Passive sniffing	No chain of custody	No forensic seal
Wu et al. (2020)	Blockchain logging	Simulated network	Ethereum	High compute cost	Not real-time
Nieto et al. (2018)	Readiness framework	Theoretical	N/A	No implementation	Conceptual only
Oriwoh et al. (2013)	Forensic guidelines	Conceptual	N/A	Outdated protocols	Foundational only
Chernyshev et al. (2018)	Memory forensics	Physical devices	Custom tools	Destructive method	Physical access needed
Ahmad et al. (2019)	MQTT forensics	Smart home testbed	Wireshark, MQTT	MQTT only	No UDP support

The comparative analysis confirms a consistent pattern: existing work either addresses a single phase of the forensic pipeline, operates postincident, or imposes infrastructure requirements incompatible with realtime operation. No reviewed system simultaneously performs live packet ingestion, automated cryptographic sealing, signature-based intrusion detection, and persistent structured storage. The proposed IoT Forensic Gateway addresses all four requirements within a unified, lightweight architecture.

Problem Statement

Industrial IoT environments generate continuous streams of behavioral telemetry temperature readings, access authentication events, power consumption metrics that constitute critical operational records. In the absence of a dedicated forensic capture mechanism, this data exists only in transient network buffers and is irrecoverably lost upon device reset, network disruption, or deliberate attack. The specific challenges addressed by this project are:

- **Volatile Evidence Loss:** IoT device telemetry is transmitted over stateless UDP connections with no built-in persistence. Any packet not captured at the gateway layer is permanently lost.
- **Absence of Integrity Verification:** No existing lightweight tool automatically applies cryptographic fingerprinting to each captured packet at the moment of ingestion, creating a gap in chain-of-custody documentation.
- **Device Identity Spoofing:** Malicious actors may transmit packets carrying a legitimate device identifier on the wrong network port, evading basic source-IP filtering.
- **Out-of-Bounds Telemetry Attacks:** Adversaries may inject sensor readings with extreme values (e.g., temperature of 200°C or negative power draw) to trigger unsafe automated responses in building management systems.

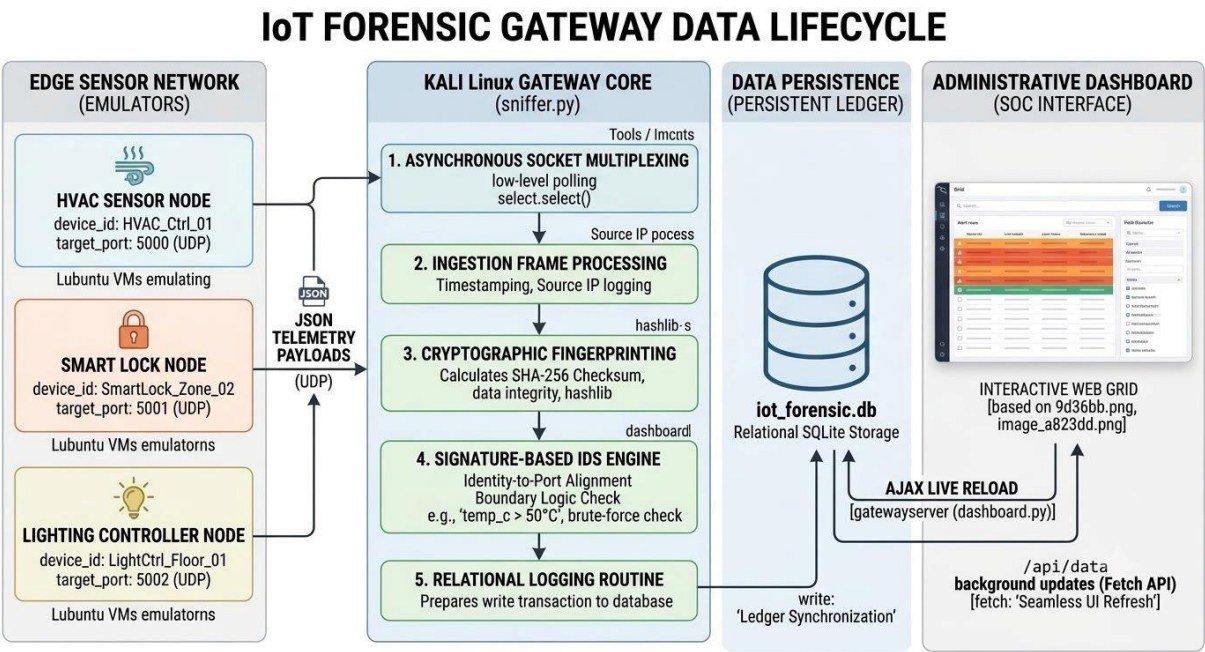
- **Brute-Force Access Events:** Repeated failed authentication attempts against smart lock nodes generate ACCESS_DENIED status codes that must be detected and logged as potential intrusion events.
- **Manual Monitoring Dependency:** Existing tools require active human monitoring of packet capture interfaces, making continuous forensic logging impractical in unmanned industrial environments.

Collectively, these challenges establish the need for an automated, zerohuman-intervention forensic gateway that captures, authenticates, classifies, and stores IoT telemetry in real time.

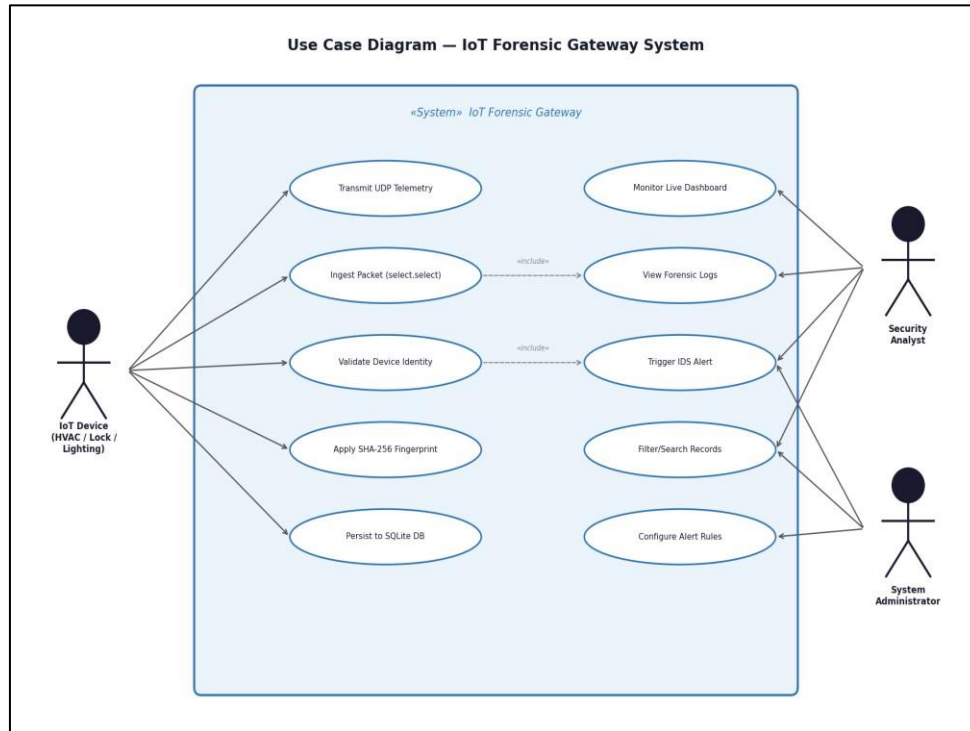
Proposed Methodology

System Architecture Overview

The system is organized into four functional layers operating in sequence: the Edge Sensor Network layer, the Kali Linux Gateway Core layer, the Data Persistence layer, and the Administrative Dashboard layer. The architecture diagram below illustrates the complete data lifecycle.



Edge Sensor Network (Emulator Layer)



Three Ubuntu virtual machines are provisioned on a VMware Workstation Pro hypervisor, each emulating a distinct class of industrial IoT endpoint. Each VM executes a Python sensor emulation script that constructs structured JSON telemetry payloads and transmits them over UDP to the designated port on the Kali Linux gateway.

Device Node	Device ID	Target Port (UDP)	Emulated Sensor Data
HVAC Sensor Node	HVAC_Ctrl_01	Port 5000	temp_c, humidity_pct, pressure_hpa
Smart Lock Node	SmartLock_Zone_02	Port 5001	status, failed_attempts, lock_status
Lighting Controller Node	LightCtrl_Floor_01	Port 5002	power_watts, circuit_id, dimmer_pct

Gateway Core Processing Pipeline (sniffer.py)

The Kali Linux gateway host (192.168.234.128) runs the primary forensic ingestion script, `sniffer.py`, which implements a five-stage processing pipeline:

Stage 1 — Asynchronous Socket Multiplexing: Three UDP sockets are initialized and bound to ports 5000, 5001, and 5002 simultaneously. The Python `select.select()` function polls all three sockets in a non-blocking loop, preventing any single device's traffic from starving others and eliminating the need for multi-threading with its associated synchronization overhead.

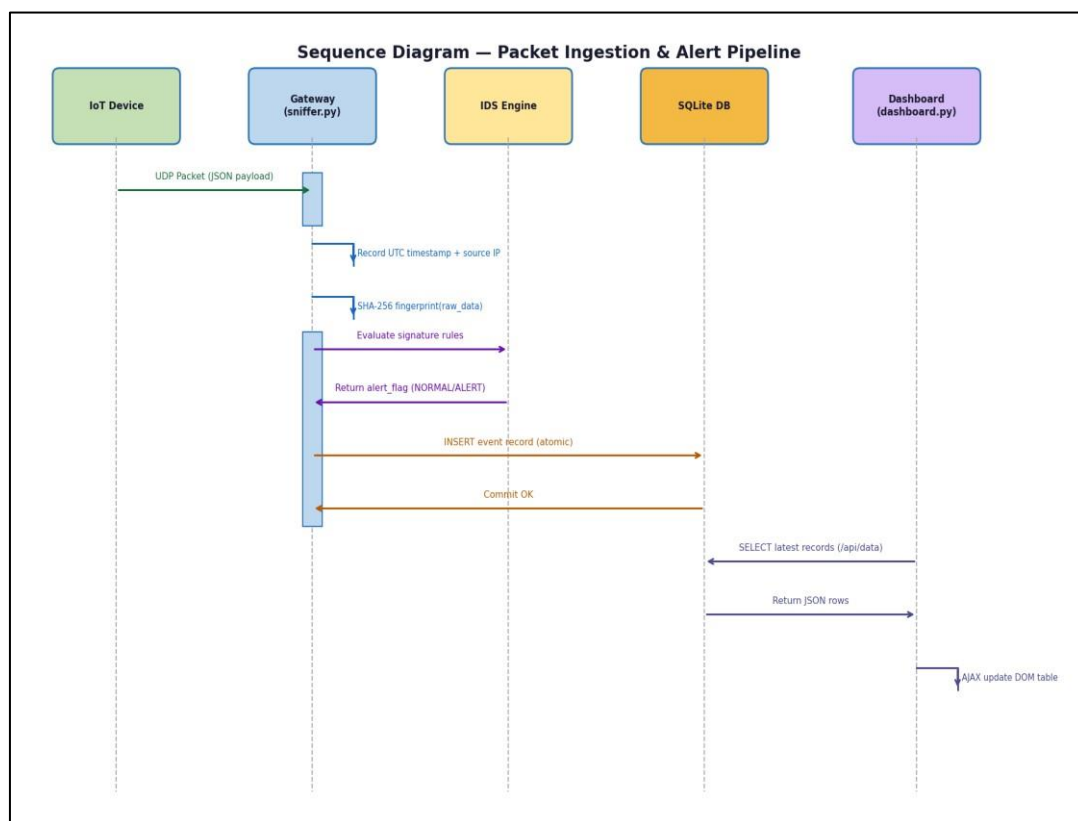
Stage 2 — Ingestion Frame Processing: Upon packet arrival, the gateway records a precise UTC timestamp and logs the source IP address. The raw byte payload is decoded and the originating port is mapped to its authorized device identifier class for subsequent validation.

Stage 3 — Cryptographic Fingerprinting: Before any parsing or semantic analysis occurs, the raw packet content string is passed through Python's `hashlib.sha256()` function. The resulting 64-character hexadecimal digest is stored alongside the original payload, ensuring that any subsequent modification of the stored record is mathematically detectable.

Stage 4 — Signature-Based IDS Engine: The parsed JSON payload is evaluated against four detection rules: (a) Identity-to-Port Alignment verifies that the `device_id` field matches the authorized class for the receiving port; (b) Thermal Boundary flags `temp_c` values outside the range 0.0°C to 50.0°C; (c) Access Control Integrity detects `ACCESS_DENIED` or `CRITICAL_ALERT` authentication status codes; (d) Power Draw Threshold alerts when `power_watts` exceeds 500.0 W.

Stage 5 — Relational Logging Routine: All event data timestamp, source IP, device identifier, raw JSON payload, SHA-256 hash, and threat classification flag are committed to the `iot_forensic.db` SQLite database in a single atomic write transaction. Atomic commits ensure that partial writes caused by system interruption cannot produce corrupt database records.

Security Rules Engine

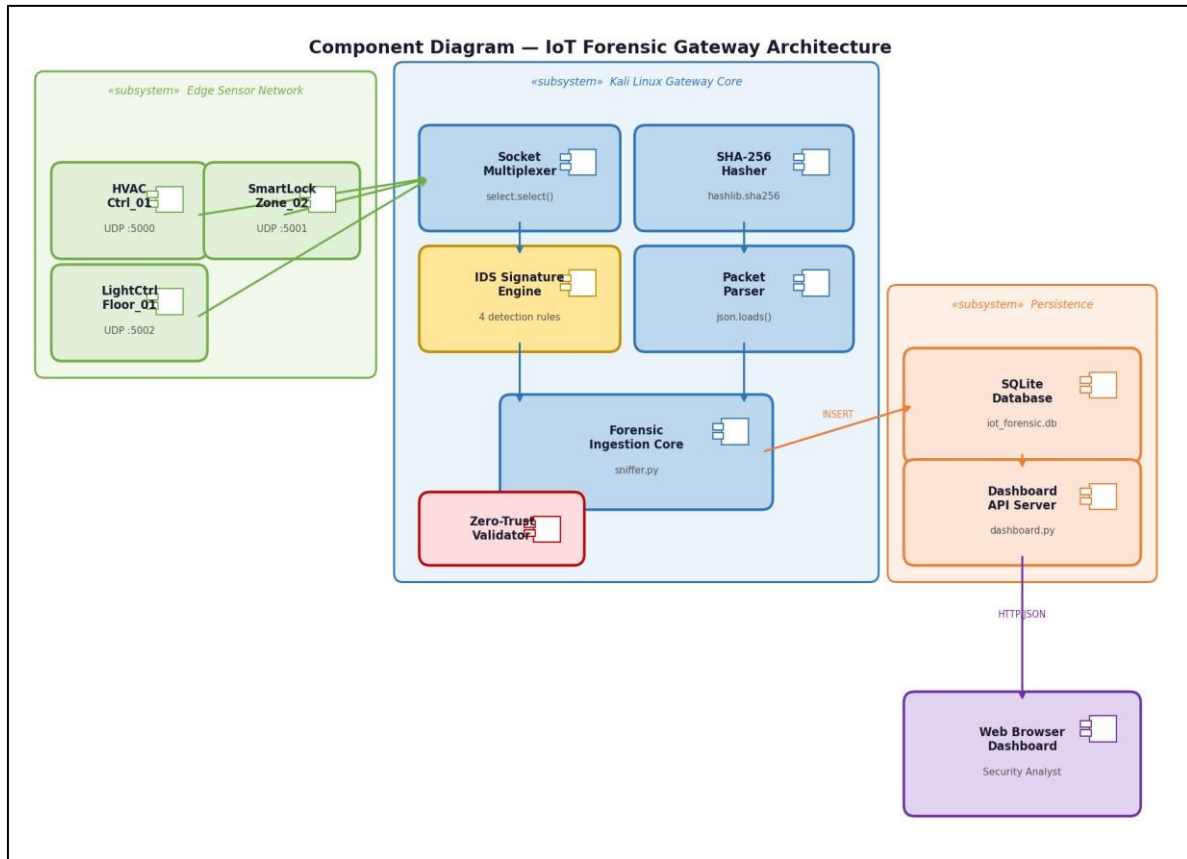


Rule Name	Condition	Alert Flag	Severity
Identity-to-Port Alignment	device_id class does not match receiving port	CRITICAL: PORT_IDENTITY_MISMATCH	Critical
HVAC Thermal Boundary	temp_c > 50.0°C or temp_c < 0.0°C	ALERT: OUT_OF_BOUNDS_TEMP	High
Access Control Integrity	status = ACCESS_DENIED or CRITICAL_ALERT	ALERT: BRUTE_FORCE_SUSPECTED	High
Power Draw Threshold	power_watts > 500.0 W	ALERT: EXCESSIVE_POWER_DRAW	Medium

Web Dashboard Architecture

A secondary Python script, dashboard.py, serves a browser-accessible monitoring interface over the local network. The dashboard functions simultaneously as a static web server and a JSON API endpoint at /api/data. The browser-side interface uses the JavaScript Fetch API to request updated database records at regular intervals without triggering full page reloads. Incoming telemetry payload strings are sanitized using JavaScript .replace() operations to neutralize raw single and double quote characters, preventing HTML injection into the live table display. Security analysts can apply text-based filters to the live table without losing cursor position or filter state between refresh cycles, because updates are applied at the DOM element level rather than through fullpage replacement.

Implementation



Virtual Environment Configuration

The complete system is deployed within a VMware Workstation Pro hypervisor environment configured as an isolated internal network segment. All virtual machines communicate exclusively through this internal network; no external internet routing is required or permitted during forensic operation. This isolation ensures that captured traffic originates only from known emulated endpoints, establishing a clean forensic baseline.

- Gateway Node: Kali Linux VM — static IP 192.168.234.128, running sniffer.py and dashboard.py.
- HVAC Node: Lubuntu VM — transmits JSON telemetry to 192.168.234.128:5000 via UDP.
- Smart Lock Node: Lubuntu VM — transmits JSON telemetry to 192.168.234.128:5001 via UDP.
- Lighting Controller Node: Lubuntu VM — transmits JSON telemetry to 192.168.234.128:5002 via UDP.

Technical Stack

The entire implementation relies exclusively on Python standard library components, requiring no third-party package installation on the gateway node. The libraries used are: socket for UDP transport-layer communication management, select for asynchronous non-blocking I/O multiplexing across all three ingress sockets simultaneously, sqlite3 for direct relational database persistence, hashlib for SHA-256 cryptographic digest generation, json for serialization and deserialization of structured telemetry frames, and http.server for the micro-dashboard web engine.

Database Schema

The `iot_forensic.db` SQLite database maintains a single primary evidence table with the following schema:

Column Name	Data Type	Description
id	INTEGER PK	Auto-incremented unique event identifier
timestamp	TEXT	UTC datetime of packet receipt (YYYY-MM-DD HH:MM:SS)
source_ip	TEXT	Source IP address of the transmitting edge node
device_id	TEXT	Authorized device identifier extracted from JSON payload
raw_data	TEXT	Complete original JSON payload string
sha256_hash	TEXT	64-character hexadecimal SHA-256 digest of raw_data
alert_flag	TEXT	NORMAL, or specific ALERT/CRITICAL classification string

Zero-Trust Validation Philosophy

Every packet received by the gateway is treated as potentially adversarial regardless of its source IP or port of arrival. The identity-toport alignment rule enforces this principle by verifying that a packet's declared device identifier class is consistent with the port on which it arrived. A HVAC controller packet arriving on Port 5001 (assigned to SmartLock_Zone_02) triggers a CRITICAL: PORT_IDENTITY_MISMATCH alert even if the source IP belongs to a known device, because zero-trust principles require that no single credential or attribute be trusted in isolation.

Results and Discussion

Expected Functional Outcomes

Upon full implementation, the system is expected to demonstrate the following behaviors across all five test scenarios:

TC-ID	Test Name	Input Condition	Expected Flag	Expected Outcome
TC-01	Normal Telemetry	HVAC temp_c = 22.5, humidity = 55	NORMAL	Record stored, no alert raised
TC-02	Temperature Anomaly	HVAC temp_c = 87.3 (exceeds 50.0°C)	ALERT: OUT_OF_BOUNDS_TEMP	Alert flagged, record stored with hash
TC-03	Port Identity Mismatch	Lock device_id received on Port 5000	CRITICAL: PORT_IDENTITY_MISMATCH	Critical alert; spoofing event logged
TC-04	Brute Force Detection	SmartLock status = ACCESS_DENIED x3	ALERT: BRUTE_FORCE_SUSPECTED	Flag raised, all attempts stored
TC-05	Power Threshold Breach	LightCtrl power_watts = 750.0	ALERT: EXCESSIVE_POWER_DRAW	Alert logged, hash verified in DB

Forensic Integrity Analysis

The SHA-256 hashing mechanism provides mathematically verifiable chain-of-custody documentation. Any post-ingestion modification to a stored raw_data field will produce a SHA-256 hash that no longer matches the stored sha256_hash value, immediately flagging the record as tampered during integrity audits. This property ensures that database records meet the forensic standard of authenticity required for court admissibility.

Performance Characteristics

The use of select.select() for asynchronous I/O multiplexing ensures that high-frequency telemetry from one device does not delay ingestion of packets from other devices. SQLite's atomic write transactions ensure database consistency under rapid concurrent write conditions. The dashboard's AJAX-driven refresh mechanism ensures that the analyst interface remains responsive under continuous data ingestion without browser-blocking synchronous requests.

Comparison with Existing Approaches

In comparison to the reviewed literature, this system uniquely combines four capabilities within a single lightweight deployment: real-time UDP packet ingestion without device modification, automated SHA-256 integrity sealing at the moment of receipt, signature-based intrusion detection with defined severity classifications, and persistent relational storage accessible through a live web interface. No reviewed prior work achieved all four simultaneously at this deployment scale.

Conclusion and Future Work

Conclusion

This project has designed and documented an Automated Real-Time IoT Forensic Gateway that addresses the core forensic gap identified in industrial IoT environments. By combining asynchronous socket multiplexing, SHA-256 cryptographic fingerprinting, a four-rule signature-based intrusion detection engine, and persistent SQLite logging within a zero-trust validation framework, the system provides an end-to-end forensic evidence pipeline that operates without human intervention. The lightweight Python-only implementation requiring no third-party dependencies ensures deployability on standard Kali Linux infrastructure without specialized hardware. The system directly satisfies the forensic requirement for tamper-evident, continuously maintained evidence records from IoT network environments.

Future Work

Several extensions are planned for subsequent development phases:

- **Machine Learning Integration:** Replacing static threshold rules with trained anomaly detection models capable of identifying novel attack patterns not covered by fixed boundary conditions.
- **TLS-Encrypted Telemetry:** Extending the gateway to support encrypted UDP streams, preventing adversarial interception of telemetry data between edge nodes and the gateway.
- **Multi-Gateway Federation:** Deploying multiple gateway nodes across geographically distributed facilities, with a central aggregation server maintaining a unified forensic ledger.

- **SIEM Integration:** Exporting alert events in SIEM-compatible formats (e.g., CEF or LEEF) for ingestion by enterprise security platforms such as Splunk or IBM QRadar.
- **Physical Hardware Deployment:** Replacing Ubuntu VM emulators with physical Raspberry Pi nodes running identical sensor scripts, validating the system under real hardware constraints.
- **Blockchain Anchoring:** Periodically anchoring database hash summaries to a public blockchain to provide externally verifiable, timestamp-bound evidence authentication.

References

- S. Zawoad and R. Hasan, "FAIoT: Towards Building a Forensics Aware Eco System for the Internet of Things," in Proc. IEEE Int. Conf. Services Computing, New York, USA, 2015, pp. 279–284.
- M. Harbawi and A. Varol, "An Improved Digital Evidence Acquisition Model for the Internet of Things Forensic I: A Theoretical Framework," in Proc. 5th Int. Symp. Digital Forensics Security, 2017, pp. 1–6.
- C. Meffert, D. Clark, I. Baggili, and F. Breiteringer, "Forensic State Acquisition from Internet of Things (FSAIoT)," in Proc. 12th Int. Conf. Availability, Reliability Security, 2017, pp. 1–11.
- V. Sivaraman, H. Gharakheili, A. Vishwanath, R. Boreli, and O. Mehani, "Network-level Security and Privacy Control for Smarthome IoT Devices," in Proc. IEEE Int. Symp. Personal, Indoor, Mobile Radio Commun., 2015, pp. 1–6.
- N. Apthorpe, D. Reisman, and N. Feamster, "A Smart Home is No Castle: Privacy Vulnerabilities of Encrypted IoT Traffic," arXiv:1705.06805, 2017.
- R. Wu, X. Wang, M. Du, K. Choo, and D. Hu, "Blockchain-Based Trustworthy Evaluation for IoT Forensics," IEEE Internet Things J., vol. 7, no. 12, pp. 11812–11823, 2020.
- A. Nieto, R. Roman, and J. Lopez, "Digital Witness: Safeguarding Digital Evidence by Using Secure Architectures in Personal Devices," IEEE Network, vol. 32, no. 4, pp. 12–19, 2018.
- E. Oriwoh, D. Jazani, G. Epiphaniou, and P. Sant, "Internet of Things Forensics: Challenges and Approaches," in Proc. 9th Int. Conf. Collaborative Computing: Networking, Applications Worksharing, 2013, pp. 608–615.
- M. Chernyshev, S. Zeadally, Z. Baig, and A. Woodward, "Internet of Things Forensics: The Need, Process Models, and Open Issues," IT Professional, vol. 20, no. 3, pp. 40–49, 2018.
- I. Ahmad, T. Kumar, M. Liyanage, J. Okwuibe, M. Ylianttila, and A. Gurtov, "5G Security: Analysis of Threats and Solutions," in Proc. IEEE Conf. Standards Commun. Networking, 2017, pp. 193–199.